伪指令和程序格式

伪指令

选择伪指令

> 使用 `.8086` 或者其他告诉汇编程序选择哪一种指令,一般**放在代码段第一条**

段定义伪指令

> 格式:
>
> ```
> segment_name segment
> ...
> segment_name ends;ends for end segment
> ```
>
> 为确定用户定义段和哪个段寄存器相关联,需要 `assume`
>
> `assume reg_name:segment_name,...,reg_name:segment_name`
>
> **`assume`指令只是指定某个段分配给哪个寄存器,并不能实际装入段寄存器,但代码段不需要**
>
> 需要:`mov ax,segment_name`和 `mov reg_name,ax`来实际装入
>
> > 还有简化段定义指令,如 `code `data
> >
> > @data在指令中表示段地址

数据定义指令:

- db:字节
- dw:字
- dd:双字
- df:六字
- dq:四字
- dt:十字

> **默认数据为十进制,且如果第一位是字母,应在前面加'0'**
>
> > 10 10H 0AH
>
> ? 表示预留空间,常被系统置为0
>
> **dup**操作符
>
> > 格式:`counts dup(contents)`
> >
> > 复制counts个contents,可以嵌套
> >
> > 如:`2 dup(1,3,2 dup(4,5))`
> >
> > `1 3 4 5 4 5 1 3 4 5 4 5`

类型属性操作符

- word ptr
- byte ptr

This和Label操作符

> `this type` 和 `name label type` 用以重新指定一个类型,且与下面的变量地址相同
>
> ```
> buf = this byte
> a dw 19,2
> buf2 label word
> b db 1,2
> ```
>
> buf和a 的地址相同但类型不同

表达式赋值伪指令

> `expression_name = Expression`
>
> `expression_name equ Expression`
>
> **=支持重复定义(即可以像C++一样多次赋值)**

地址计数和定位伪指令

> `$` 地址计数器
>
> 每段开始,地址计数器都初始化为0,每执行一条指令,地址计数器增加
>
> `org` 用来设置地址计数器的值
>
> > `org n` 表示下一个字节的存放地址为n

even伪指令

> `even` 使下一个变量或指令开始于偶数地址
>
> 16位变量,偶数只需要一次读写,而奇数需要两次读写

align伪指令

> `align n` 使下一个变量的地址从4的倍数开始,可以保证双字数组边界从4的倍数开始
>
> n必须是2的幂

基数控制指令

> `.radix expression` 把默认基数改成2~16范围内任何基数
>
> expression用十进制表示

过程定义伪指令

```
data segment
    string db 'hello word$'
data ends
code segment
    assume cs:code,ds:data
main proc far;对于系统程序来说是DOS调用的远过程,因此是far
    mov ax,data
    mov ds,ax
    mov dx,offset string
    mov ah,9
    int 21h
    mov ah,4ch
```

```
    int 21h
main endp
code ends
end main; 告诉程序起点是main
```

表达式与操作符

算术和逻辑和逻辑移位操作符

> +,-,*,/,mod
>
> 其结果必须要有明确的物理意义
>
> 如:`mov ax,da1*4` da1为地址,地址*4没意义
>
> 如:`mov ax,val*4/2` val为具体值,有意义
>
> 逻辑和逻辑移位操作符:只能用于数字表达式中
>
> `expression 操作符 number`
>
> 如: `mov ax, bx and offH` 不行,bx and 必须指令
>
> 如: `mov ax,val and 0f0H` 可以

关系操作符

EQ NE LT GT LE GE

关系操作符的两个操作数必须都是数字,或者同一段的存储器地址,真为FFFFH,假为0000H

数值回送操作符

- type

  > `type expression`
  >
  > 如果表达式是变量,则汇编回送变量代表的类型
  >
  > > db 1 dw 2 dd 4 df 6 dq 8 dt 10
  >
  > 如果表达式是标号,则回送标号类型
  >
  > > near -1 far -2

- length

  > 变量用 `dup` 的回送总变量数,其他为1,嵌套dup不算
  >
  > `arr dw 4 dup(3)` 回送 4
  >
  > `var dw 12,34,45` 回送1

- size

  > 变量用 `dup` 的回送总字节数,其他为单变量的字节数,嵌套的dup不算
  >
  > `arr dw 4 dup(3)` 回送 8
  >
  > `var dw 12,34,45` 回送2

- offset

  > `offset var或label`

> 回送变量或标号的偏移地址

- seg

> `seg var或label`
>
> 回送变量或标号的段地址

exe和com文件

PSP:程序段前缀

程序在执行前调入内存时,由DOS确定装入的起始地址,并在此建立一个PSP,再装入程序.

PSP的长度为256字节,其中[PSP:0]存放的是 `int 20h` 用以程序返回的中断调试指令

> DS=ES=PSP地址
>
> CS:IP=程序执行的地址
>
> SS:SP=堆栈段的栈底地址

为何要额外给段寄存器赋值的原因:程序在装入前无法确定在内存的物理地址,link后的data只是一个相对地址,在装入后DOS给data转化为绝对地址,之后的 `mov ax,data  mov ds,ax` 才能起作用

使用ret指令结束程序

```
data segment
    string db 'hello word$'
data ends
code segment
    assume cs:code,ds:data
main proc far
    push ds;压入PSP地址
    xor ax,ax;归零
    push ax;指明ret后执行PSP的第一条地址
    ;必做不然ret会出错

    mov ax,data
    mov ds,ax
    mov dx,offset string
    mov ah,9
    int 21h

    ret
main endp
code ends
end main; 告诉程序起点是main
```

> 程序开始向堆栈压入DS和0,在ret结束程序后,把0弹出到IP,把PSP弹出到CS,然后就会执行[PSP:0]的指令,即执行 `int 20h`
>
> 如果是段内调用只会弹出一个字到IP,如果段间还要再弹出一个字到CS (这是后续子程序设计的Call)

COM文件不做过多说明,在link后加入/t即可生成COM文件(`link hello/t`),相比EXE占用空间更小

> 同名COM和EXE,只有完整输入(.exe)EXE的文件名才执行,否则执行COM文件

分支和循环程序设计

单分支

```
;双字长 数 放在dx和ax中,求绝对值
code segment
    assume cs:code
main proc far
    push ds
    xor ax,ax
    push ax

    test dx,8000h
    jz exit
    not ax
    not dx
    add ax,1
    adc dx,0
exit:
    ret
main endp
code ends
end main
```

复合分支

```
;输入一位十六进制数,转换为十进制输出
code segment
    assume cs:code
main proc far
    push ds
    xor ax,ax
    push ax

    mov ah,1
    int 21h
    cmp al,30h
    jl exit
    cmp al,39h
    jle dig
    ;判断数字
    cmp al,41h
    jl exit
    cmp al,46h
    jle print
    ;判断大写
    cmp al,61h
    jl exit
    cmp al,66h
    jg exit
    sub al,31h;小写改成数字
    jmp out1
    ;小写处理
print:
    sub al,11h;大写改成数字
```

```
out1:;输出1,因为是字母肯定是10以上
    mov dl,31h
    mov ah,2
    push ax;int21h会改变ax值,存入栈保存
    int 21h
    pop ax
dig:
    mov dl,al
    mov ah,2
    int 21h
exit:
    ret
main endp
code ends
end main
```

多分支

```
;输入一个数字,转移到四个不同分支
code segment
    assume cs:code,ds:code
main proc far
    push ds
    xor ax,ax
    push ax

    mov ax,code
    mov ds,ax
    mov ah,7
    int 21h
    cmp al,31h
    jl exit
    cmp al,34h
    jg exit
    mov dl,al
    mov bl,al
    sub bl,31h;转成数字
    shl bl,1;乘 2 因为是word
    mov bh,0
    jmp branch[bx]
r1:
    mov ah,2
    int 21h
    jmp exit
r2:
    mov ah,2
    int 21h
    jmp exit
r3:
    mov ah,2
    int 21h
    jmp exit
r4:
    mov ah,2
    int 21h
```

```
    jmp exit
exit:
    ret
branch:
    dw r1
    dw r2
    dw r3
    dw r4
main endp
code ends
end main
```

计数循环

```
;把bx二进制转成十六进制
code segment
    assume cs:code
main proc far
    push ds
    xor ax,ax
    push ax

    mov cx,4
shift:
    ;因为cl被占用了,所以只能多次使用rol循环左移
    rol bx,1
    rol bx,1
    rol bx,1
    rol bx,1
    mov al,bl
    and al,0fh;取低四位
    add al,30h;变字
    cmp al,39h
    jle dig
    add al,7;改成字母
dig:
    mov dl,al
    mov ah,2
    int 21h
    loop shift
    ret
main endp
code ends
end main
```

条件循环程序

```
;逻辑尺,
code segment
    assume cs:code
main proc far
    push ds
    xor ax,ax
    push ax
```

```
    mov bx,0
    mov cx,8
inlog:
    mov ah,1
    int 21h
    cmp al,30h
    jb exit
    cmp al,31h
    ja exit
    sub al,30h
    shl bl,1
    add bl,al; 左移存入逻辑尺
    loop inlog
    mov ah,2
    mov dl,10
    int 21h;输出换行
inchr:
    mov ah,1
    int 21h
    cmp al,13
    je exit;回车退出
    mov dl,al
    rol bl,1
    jnc k30
    inc dl
    jmp putc
k30:
    dec dl
putc:
    mov ah,2
    int 21h
    jmp inchr
exit:
    ret
main endp
code ends
end main
```

条件计数循环

```
;输入一串字符串,再输入一个字符并查找是否出现
data segment
    buffer db 16,?,16 dup(?),13,10,'$'
    inputs db 13,10,"input string:$"
    getc db 13,10,"input char:$"
    output db 13,10,'not found$'
data ends
code segment
    assume cs:code,ds:data
main proc far
    push ds
    xor ax,ax
    push ax

    mov ax,data
```

```
    mov ds,ax
    lea dx,inputs
    mov ah,9
    int 21h
    lea dx,buffer
    mov ah,10;0Ah
    int 21h
    lea dx,getc
    mov ah,9
    int 21h
    mov ah,1;输入字符
    int 21h
    mov bl,al;传入bl
    lea di,buffer+1;指向缓冲区前一位
    mov cl,buffer+1;设置比较次数
    mov ch,0
seek:
    inc di
    cmp bl,[di];比较
    loopne seek;循环
    jne nof;不相等则没找到
    mov dl,10
    mov ah,2
    int 21h
    lea dx,buffer+2
    mov ah,9
    int 21h
    jmp exit
nof:
    lea dx,output
    mov ah,9
    int 21h
exit:
    ret
main endp
code ends
end main
```

多重循环

就是多个循环嵌套,合理使用寄存器即可

子程序设计

Call

> 格式: `call [near/far ptr] dst`
>
> 先向堆栈保存返回地址,再实现程序转移
>
> > 对于段内调用,只是保存IP
> >
> > 对于段间调用,还有先保存CS
>
> 属性说明可以省略

进入子程序执行后,必然要使用寄存器,寄存器原有的值就被改变.如果这些寄存器的值返回主程序后还要使用,则需要在进入子程序后保存这些值,并在退出子程序前恢复这些值.--即所谓的在子程序中对主程序的现场实施保护和恢复

```
push ax
push bx
...
pop bx
pop ax
```

子程序的参数传递

> 入口参数,返回参数
>
> 值传递,地址传递

- 寄存器
- 变量
- 地址表
- 堆栈
- 结构变量

  > ```
  > 结构名 struc
  >  变量 db/dw/... 值
  >  ...
  >  变量 db/dw/... 值
  > 结构名 ends
  > ```
  >
  > 访问方式:
  >
  > ```
  > mov si,1
  > lea bx,struc_name
  > mov al,struc_name.变量[si];结构的某个变量的第几字节
  > mov al,[bx].变量[si];结构的某个变量的第几字节
  > mov al,struc_name+n*结构长度.变量[si];第n个结构的某个变量的第几字节(假使结构名是一串结构的起始地址)
  > ```

多模块

本模块定义,只在本模块使用叫局部模块

本模块定义,在其他模块使用叫外部符号

`extern 符号:类型...`

> 指出本模块的符号名是其他模块定义的
>
> - 如果是变量要指明是byte还是...
> - 如果是标号或过程名,要指明near或far

`public 符号`

多模块单独汇编,在进行连接,连接时主模块在前

如:`link modelMain + modelSub`

其他深入内容请看 **汇编语言程序设计(第二版):ISBN 978-7-115-44297-0**